Wirtualna i rozszerzona rzeczywistość

Wykład 7

Cyfrowy bliźniak, ROS 2 i teleoperacja XR

dr inż. Mateusz Pomianek

Automatyka i Robotyka, studia II stopnia, 3 ECTS

Katedra Informatyki i Automatyki

Rok akademicki 2026/2027

Wersja 1.4.2 14.10.2026

Plan i rytm spotkania

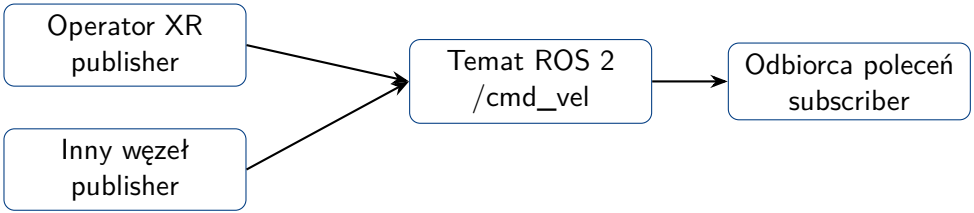
1. Rzeczywisty problem integracji ROS 2 i XR
2. ROS 2 jako warstwa systemowa
3. Demonstracja: kontrola świeżości danych
4. Przerwa
5. Teleoperacja, synchronizacja i bezpieczeństwo
6. Demonstracja: od wiadomości ROS 2 do widoku XR
7. Domknięcie: architektura odporna na błędne dane

Jedna przerwa oddziela podstawy ROS 2 od teleoperacji, bezpieczeństwa i studium integracji ROS 2 z XR.

Po tym wykładzie potrafisz...

- prześledzić przepływ poleceń i stanu między ROS 2 a aplikacją XR,
- zidentyfikować nadawców tematu i granice samego mechanizmu publish-subscribe,
- sprawdzić świeżość wiadomości ze zgodnym zegarem i znacznikiem czasu,
- rozróżnić regułę aplikacyjną, zabezpieczenia SROS2 i niezależne funkcje bezpieczeństwa robota.

Temat /cmd_vel może mieć więcej niż jednego nadawcę



Co wynika z architektury?

Sam temat ROS 2 nie przyznaje wyłączności jednemu nadawcy. Dostęp do grafu i prawo publikowania trzeba zabezpieczyć niezależnie od logiki interfejsu XR.

Nadawców sprawdzamy poleceniem ROS 2

Izolowane środowisko demonstracyjne

```
ros2 topic info -v /cmd_vel
```

Odczyt grafu

Liczba publisherów, identyfikacja węzłów, typ wiadomości i profile QoS.

Czego wynik nie dowodzi?

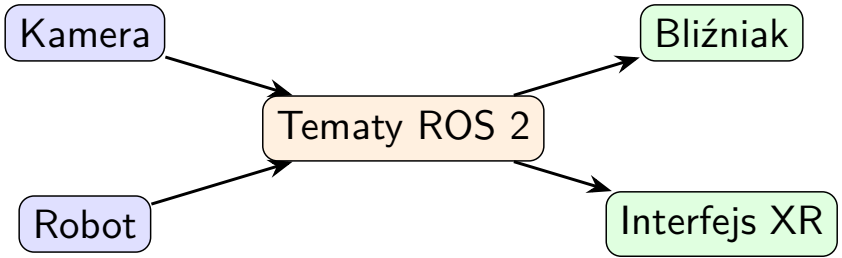
Tożsamości uprawnionego operatora ani bezpiecznego sterowania obiektem fizycznym.

Węzły i tematy: model publish-subscribe

- **Węzeł** to niezależny proces realizujący konkretną funkcję.
- **Temat** przenosi strumień wiadomości między publikującymi i subskrybentami.
- Nadawca nie musi znać odbiorcy, a wielu odbiorców może słuchać tego samego tematu.

Konsekwencja

Otwarta architektura ułatwia integrację, ale bez dodatkowych polityk nie odpowiada na pytanie, kto ma prawo publikować polecenia sterujące.



QoS: różne dane potrzebują różnych gwarancji

Sterowanie

- RELIABLE,
- kontrola Deadline,
- kontrola Liveliness,
- ważniejsza pewność dostarczenia.

Strumień obrazu

- często BEST_EFFORT,
- pojedyncza utracona klatka jest akceptowalna,
- niższe opóźnienie ważniejsze od retransmisji.

Czas: kiedy dane powstały, a nie kiedy dotarły

Znacznik czasu

Wiadomości sensoryczne powinny przenosić czas powstania pomiaru. Dzięki temu odbiorca może odróżnić opóźnioną wiadomość od nowego pomiaru.

Czas symulacyjny

W virtual commissioning czas symulacji może płynąć inaczej niż zegar systemowy. Węzły muszą korzystać z właściwego źródła czasu ROS 2.

tf2: transformacje jako historia w czasie

- tf2 utrzymuje drzewo układów współrzędnych,
- każda transformacja ma znacznik czasu,
- bufor pozwala pytać o relacje przestrzenne z określonego momentu,
- to kluczowe przy opóźnionych danych z kamer i sensorów.

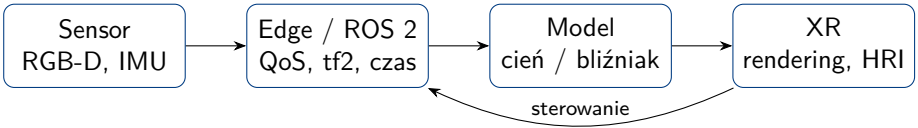
Pytanie systemowe

Gdzie była końcówka robota względem świata w chwili, gdy kamera wykonała konkretną klatkę?

sensor_msgs: wspólny słownik systemu

CameraInfo	parametry wewnętrzne kamery i dystorsja,
Image / PointCloud2	obraz i geometria głębi,
Imu	akcelerometr i żyroskop,
JointState	pozycje i prędkości przegubów robota.

Sensor - edge - model - XR



Idea przewodnia

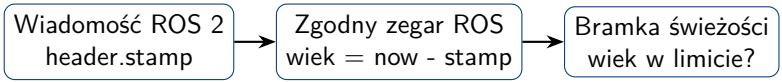
Awaria lub opóźnienie w dowolnym ogniwie może unieważnić poprawne działanie pozostałych.

Przykład kodu ROS 2: kontrola świeżości

1. Odczytaj znacznik czasu wiadomości ze struktury `header.stamp`.
2. Porównaj go ze zgodnym zegarem ROS 2, uwzględniając czas symulacyjny.
3. Odrzuć dane przyszłe lub starsze niż przyjęty limit.
4. Nie dopuszczaj polecenia, jeżeli krytyczny stan jest nieaktualny.

Przykłady Python: `snippets/stale_guard.py` i `snippets/publisher.py`.

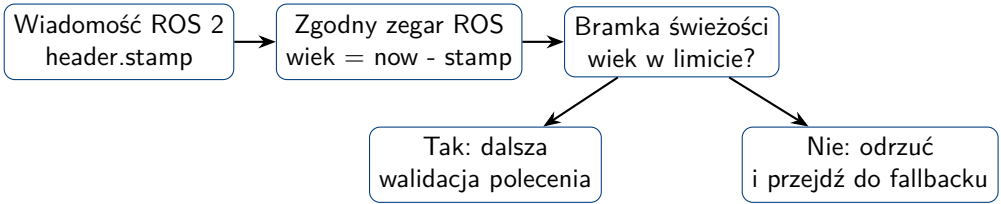
Przed ruchem robota sprawdzamy wiek informacji



Rzeczywista implementacja

`header.stamp` musi odnosić się do tego samego źródła czasu co odbiorca. Próg maksymalnego wieku jest wymaganiem danej aplikacji, nie stałą narzuconą przez ROS 2.

Przed ruchem robota sprawdzamy wiek informacji



Rzeczywista implementacja

`header.stamp` musi odnosić się do tego samego źródła czasu co odbiorca. Próg maksymalnego wieku jest wymaganiem danej aplikacji, nie stałą narzuconą przez ROS 2.

Przerwa 5 min

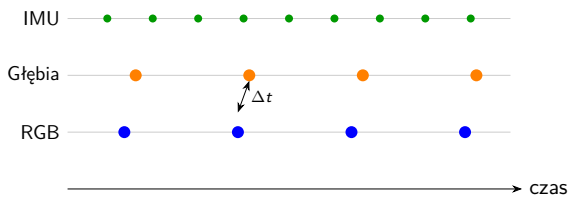
Synchronizacja: dane nie przychodzą razem

Problem

Kamera RGB, mapa głębi i IMU publikują z różną częstotliwością i różnym opóźnieniem. Najnowsze dostępne wiadomości nie muszą opisywać tego samego momentu.

Rozwiązanie

ExactTime wymaga zgodnych znaczników czasu. ApproximateTime pozwala dobrać wiadomości mieszczące się w zadanej tolerancji czasowej.



Stale data: problem całego systemu

- obraz może być świeży, a stan robota stary,
- IMU może działać, gdy kamera chwilowo milczy,
- każdy krytyczny węzeł powinien sprawdzać wiek wszystkich danych wejściowych,
- brak świeżych danych powinien prowadzić do jawnego stanu awaryjnego, nie do cichego używania starej wartości.

Teleoperacja: dwa kierunki opóźnienia

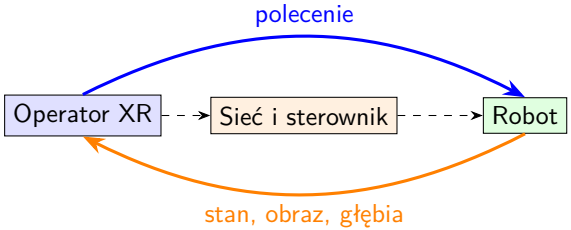
Operator do robota

Polecenie przechodzi przez aplikację, sieć i warstwę sterowania.

Robot do operatora

Stan robota, obraz i głębia wracają osobnym kanałem.

Wniosek: opóźnienie obu kanałów sumuje się w pętli sprzężenia zwrotnego; lokalna płynność XR nie wystarcza.



Virtual commissioning a cyfrowy bliźniak

- virtual commissioning służy do walidacji logiki sterowania na modelu symulacyjnym,
- może używać prawdziwego oprogramowania sterującego jeszcze przed powstaniem fizycznego stanowiska,
- przed uruchomieniem sprzętu mówimy raczej o modelu lub pre-bliźniaku niż o pełnym operacyjnym bliźniaku,
- pełny bliźniak wymaga powiązania z działającym obiektem fizycznym i bieżącym przepływem danych.

SROS2: trzy filary ochrony

Uwierzytelnianie potwierdza tożsamość uczestnika komunikacji.

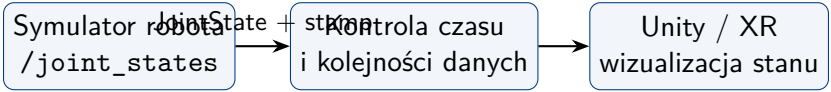
Kontrola dostępu określa, który węzeł może publikować i subskrybować konkretne tematy.

Szyfrowanie chroni dane przesyłane przez sieć przed podsłuchem i modyfikacją.

Konsekwencja architektoniczna

Samo szyfrowanie nie wystarczy. Kluczowe jest także ograniczenie prawa publikacji na temat sterujący robotem.

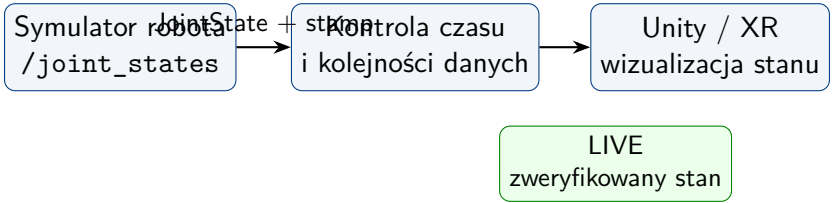
Strumień JointState nie gwarantuje świeżego widoku XR



Reguła

Widoczność modelu nie świadczy o aktualności danych. Status powinien wynikać ze znacznika czasu i konfiguracji strumienia.

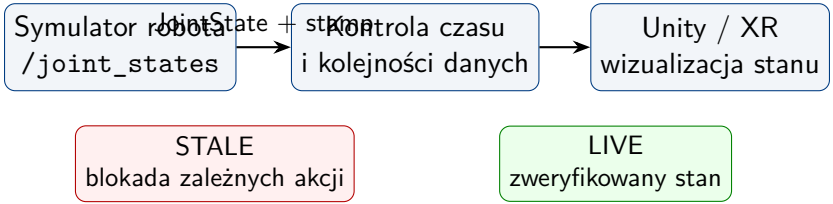
Strumień JointState nie gwarantuje świeżego widoku XR



Reguła

Widoczność modelu nie świadczy o aktualności danych. Status powinien wynikać ze znacznika czasu i konfiguracji strumienia.

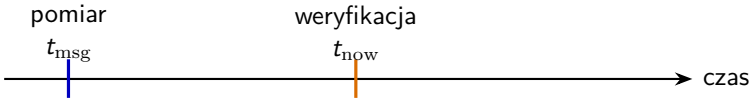
Strumień JointState nie gwarantuje świeżego widoku XR



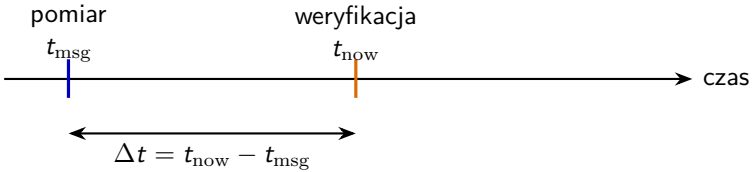
Reguła

Widoczność modelu nie świadczy o aktualności danych. Status powinien wynikać ze znacznika czasu i konfiguracji strumienia.

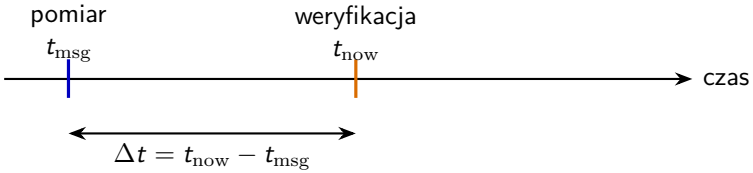
Świeżość to nierówność, nie intuicja operatora



Świeżość to nierówność, nie intuicja operatora



Świeżość to nierówność, nie intuicja operatora



LIVE: $0 \leq \Delta t \leq T_{\max}$

STALE: $\Delta t > T_{\max}$ lub niespójny czas

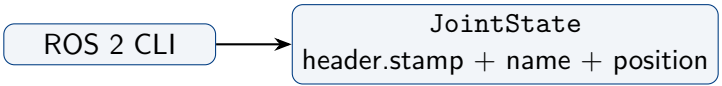
Terminal ROS 2: od nazwy tematu do prawdziwej wiadomości

Polecenia tylko do odczytu

```
ros2 topic list
ros2 topic info -v /joint_states
ros2 topic echo /joint_states --once
```

Co sprawdzić

- typ wiadomości,
- publikujący węzły,
- profile QoS,
- znacznik czasu,
- nazwy stawów i wartości.



Kod ROS 2: odczyt wieku wiadomości JointState

```
    def __init__(self):
        super().__init__("joint_state_age")
        self.declare_parameter("max_age_s", -1.0)
        self.create_subscription(
            JointState, "/joint_states", self.inspect,
            qos_profile_sensor_data,
        )

    def inspect(self, msg):
        limit = float(self.get_parameter("max_age_s").
     value)
        stamp = Time.from_msg(msg.header.stamp)
        age = (self.get_clock().now() - stamp).
     nanoseconds * 1e-9
        fresh = limit > 0 and stamp.nanoseconds > 0 and 0
      <= age <= limit
        self.get_logger().info(f"JointState: {'LIVE' if
     fresh else 'STALE'}")
```

Subskrypcja
`/joint_states` tylko do odczytu.

Bramka
Ważny znacznik czasu, zgodny zegar, dodatni próg `max_age_s`.

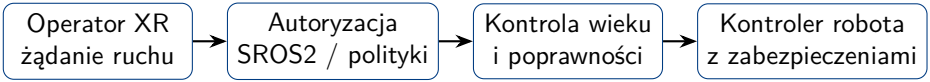
Bez napędu
Przykład nie publikuje poleceń sterujących.

Zrzut: rzeczywista wiadomość JointState i jej źródło



Miejsce na rzeczywisty zrzut ekranu: Terminal ROS 2: `ros2 topic info -v /joint_states` i fragment `ros2 topic echo /joint_states --once` z faktycznym `header.stamp`; bez danych poufnych.

Polecenie XR nie trafia bezpośrednio na napęd



Stan robota wraca do XR osobnym kanałem z własnym znacznikiem czasu i kontrolą świeżości.

Rozdzielenie odpowiedzialności

SROS2 służy ochronie komunikacji, aplikacja odrzuca nieaktualne dane, a niezależne funkcje bezpieczeństwa sterownika odpowiadają za ograniczenie ryzyka fizycznego.

Trzy niezależne kryteria odbioru interfejsu ROS 2–XR

1. **Dostęp**: nieautoryzowany węzeł nie może wykonać komendy sterującej.
2. **Czas**: wiadomość stara, przyszła albo z niespójnego zegara nie może uruchomić akcji.
3. **Stan fizyczny**: zatrzymanie awaryjne i ograniczenia napędu działają niezależnie od interfejsu XR.

Dowód

Testy przeprowadza się najpierw w symulacji lub środowisku izolowanym. Dopiero zweryfikowany projekt można poddać odrębnej analizie ryzyka i procedurze dopuszczenia do testów sprzętowych.

Przykładowe pytania egzaminacyjne

1. Jak ustalić za pomocą narzędzi ROS 2, kto publikuje i odbiera wiadomości danego tematu?
2. Jak sprawdzać znacznik czasu i spójność układów odniesienia danych przesyłanych z ROS 2 do XR?
3. Jak dobrać QoS do danych sensorowych, a jak do poleceń sterujących, uwzględniając charakter transmisji?
4. Dlaczego SROS2, odrzucanie przeterminowanych poleceń i sprzętowe funkcje bezpieczeństwa pełnią różne role?

Bibliografia: kluczowe źródła

ROS 2 i transmisja

- ROS 2 Documentation, *About Quality of Service Settings*.
- ROS 2 Documentation, *tf2: Transformations*.
- ROS 2 Documentation, *SROS2*.

Integracja i robotyka

- M. Quigley i in., *ROS: an Open-Source Robot Operating System*, 2009.
- W. Meeussen, *REP-105: Coordinate Frames for Mobile Platforms*.
- Unity Technologies, *ROS–TCP Connector*, dokumentacja.

Pełna bibliografia: `../../bibliography.bib`.

Kolejne materiały

← Wykład 6

Laboratorium 7